

School Management App

ORM Kapcsolatok és \$casts – Fejlesztői dokumentáció

Laravel 12 + Eloquent ORM | 2026

1. Mi az Eloquent ORM?

Az Eloquent a Laravel beépített ORM (Object-Relational Mapper) rendszere. Feladata, hogy az adatbázis tábláit PHP osztályokként (modellekként) kezelje, és ne kelljen nyers SQL lekérdezéseket írni.

Nyers SQL	Eloquent ORM
SELECT * FROM students WHERE id = 1;	Student::find(1);
SELECT * FROM students WHERE class_id = 5;	Student::where('class_id', 5)->get();
INSERT INTO students (name, email) VALUES (...)	Student::create(['name' => ..., 'email' => ...]);
DELETE FROM students WHERE id = 1;	\$student->delete();

2. ORM Kapcsolatok

A kapcsolatok megmondják az Eloquentnek, hogy az egyes táblák hogyan viszonyulnak egymáshoz. A projektben három kapcsolattípust használunk:

2.1 belongsTo – "én tartozom valakihez"

A belongsTo mindig annál a modellnél van, amelyik tartalmazza a foreign key-t. A students táblában van egy class_id mező, ezért a Student modell "tartozik" egy SchoolClass-hoz.

```
class Student extends Model
{
    public function schoolClass(): BelongsTo
    {
        return $this->belongsTo(SchoolClass::class, 'class_id');
    }
}

// Használat:
$student = Student::find(1);
$student->schoolClass->name; // '10A'
```

Ökölszabály: ha a táblában van egy foreign key mező (pl. class_id), az a modell belongsTo-t kap.

2.2 hasMany – "nekem sok van belőlem"

A hasMany mindig a másik oldalon van – ahol nincs foreign key, de hivatkoznak rá. Egy osztálynak sok diákja van, ezért a SchoolClass modell hasMany kapcsolattal rendelkezik a Student felé.

```
class SchoolClass extends Model
{
    public function students(): HasMany
    {
        return $this->hasMany(Student::class, 'class_id');
    }
}

// Használat:
$class = SchoolClass::find(1);
$class->students;           // az összes diák ebben az osztályban
$class->students->count(); // hány diák van
```

2.3 hasOne – "nekem pontosan egy van"

Akkor használjuk, ha a kapcsolat egy-az-egyhez. A belongsTo tükörképe, de ott ahol nincs foreign key. Például egy osztálynak egy órarendje van.

```
class SchoolClass extends Model
{
    public function timetable(): HasOne
    {
        return $this->hasOne(Timetable::class, 'class_id');
    }
}

// Használat:
$class->timetable->day; // 'monday'
```

2.4 Kapcsolatok összefoglalása

Kapcsolat típusa	Mikor használjuk	Foreign key helye
belongsTo	Ez a modell hivatkozik másikra	Ebben a táblában van
hasMany	Másik modell hivatkozik erre, többes	A másik táblában van

hasOne	Másik modell hivatkozik erre, egyes	A másik táblában van
--------	-------------------------------------	----------------------

2.5 Minden kapcsolat a projektben

Modell	Kapcsolat	Másik modell	Magyarázat
Student	belongsTo	SchoolClass	Egy diák egy osztályba jár
Student	hasMany	Guardian	Egy diáknak több gondviselője lehet
Student	hasMany	Attendance	Egy diáknak több jelenléti bejegyzése van
Student	hasMany	ExamResult	Egy diáknak több vizsgaeredménye van
Student	hasMany	Fee	Egy diáknak több díja lehet
Student	hasMany	LibraryIssue	Egy diák több könyvet kölcsönözhet
Teacher	hasMany	Subject	Egy tanár több tantárgyat taníthat
Teacher	hasMany	SchoolClass	Egy tanár több osztály osztályfőnöke lehet
Teacher	hasMany	Attendance	Egy tanárnak több jelenléti bejegyzése van
Teacher	hasMany	Timetable	Egy tanár több órát tarthat
SchoolClass	belongsTo	Teacher	Egy osztálynak egy osztályfőnöke van
SchoolClass	hasMany	Student	Egy osztályban több diák van
SchoolClass	hasMany	Timetable	Egy osztálynak több órarend bejegyzése van
SchoolClass	hasMany	Exam	Egy osztálynak több vizsgája lehet
Subject	belongsTo	Teacher	Egy tantárgyat egy tanár tanít
Guardian	belongsTo	Student	Egy gondviselő egy diákhoz tartozik
Exam	hasMany	ExamResult	Egy vizsgán több diák írhat
ExamResult	belongsTo	Exam	Egy eredmény egy vizsgához tartozik
ExamResult	belongsTo	Student	Egy eredmény egy diákhoz tartozik
LibraryBook	hasMany	LibraryIssue	Egy könyvet többen kölcsönözhetnek
Transport	hasMany	TransportStop	Egy járatnak több megállója van
Transport	hasMany	TransportStudent	Egy járatot több diák használ
Notification	hasMany	NotificationRead	Egy értesítést többen olvashatnak

3. Eager Loading – N+1 probléma

Az egyik leggyakoribb teljesítményhiba Laravel projektekben az N+1 probléma. Ezt Eager Loading-gal kerüljük el.

3.1 A probléma – N+1 lekérdezés

```
// ROSSZ - N+1 probléma
$students = Student::all(); // 1 lekérdezés

foreach ($students as $student) {
    echo $student->schoolClass->name;
    // minden diáknál 1 újabb lekérdezés fut!
}

// Ha 100 diák van → 101 lekérdezés fut le!
```

3.2 A megoldás – Eager Loading with()

```
// JÓ - Eager Loading
$students = Student::with('schoolClass')->get(); // 2 lekérdezés
összesen

foreach ($students as $student) {
    echo $student->schoolClass->name;
    // nincs újabb lekérdezés!
}

// Akárhány diák → mindig csak 2 lekérdezés!
```

3.3 Több kapcsolat egyszerre

```
// Több kapcsolat betöltése egyszerre:
$students = Student::with([
    'schoolClass',
    'guardians',
    'examResults',
])->get();

// Beágyazott kapcsolatok:
$classes = SchoolClass::with('students.guardians')->get();
// Betölti az osztályokat, a diákjaikat, és a diákok gondviselőit
```

4. A \$casts tömb

Az adatbázis mindent szöveggént (string) ad vissza PHP-nak. A \$casts megmondja az Eloquentnek, hogy egyes mezőket automatikusan konvertáljon a megfelelő PHP típusra – mentéskor és olvasáskor egyaránt.

4.1 Miért kell a \$casts?

```
// $casts NÉLKÜL:
$student->date_of_birth;           // '2005-03-15' → sima string, semmi
mást nem tud
$fee->amount;                       // '15000.50' → string, számolni
nem lehet vele

// $casts-SZAL:
protected $casts = [
    'date_of_birth' => 'date',
    'amount'        => 'decimal:2',
];

$student->date_of_birth->age;        // 19 → Carbon objektum, mindent
tud!
$student->date_of_birth->format('Y. m. d.');// '2005. 03. 15.'
$fee->amount + 500;                  // 15500.50 → valódi szám
```

4.2 date – dátum cast

A date típusú mezőket Carbon objektummá alakítja. A Carbon egy PHP könyvtár, amellyel könnyen lehet dátumokkal számolni.

```
protected $casts = [
    'date_of_birth' => 'date',    // Student, Teacher
    'exam_date'     => 'date',    // Exam
    'due_date'      => 'date',    // Fee, LibraryIssue
    'joining_date'  => 'date',    // Teacher
];

// Amit ezzel tudunk csinálni:
$fee->due_date->isPast();           // lejárt-e már? → true/false
$fee->due_date->diffInDays(now());  // hány nap van még hátra?
$teacher->joining_date->format('Y. m. d.');// '2020. 09. 01.'
$student->date_of_birth->age;       // életkor számítása
```

4.3 datetime – dátum és idő cast

Ugyanaz mint a date, de az időt is tárolja. Szintén Carbon objektum lesz belőle.

```
protected $casts = [
    'sent_at' => 'datetime',    // Notification
    'read_at' => 'datetime',    // NotificationRead
];

// Amit ezzel tudunk csinálni:
$notification->sent_at->format('Y-m-d H:i');    // '2026-06-05 14:30'
$notification->sent_at->diffForHumans();        // '2 órája'
$notification->sent_at->isToday();              // ma küldték-e?
```

4.4 datetime:H:i – idő formátum cast

Csak az időt tárolja és formázza H:i formátumban (óra:perc). Az órarendnél és vizsgáknál használjuk.

```
protected $casts = [
    'start_time' => 'datetime:H:i',    // Timetable, Exam
    'end_time'    => 'datetime:H:i',
    'pickup_time' => 'datetime:H:i',    // TransportStop
];

// Amit ezzel tudunk csinálni:
$timetable->start_time;    // '08:00'
$timetable->end_time;       // '09:30'
```

4.5 decimal:2 – pénzügyi szám cast

Pénzügyi számokhoz mindig decimal típust használunk, soha nem float-ot. A float kerekítési hibákat okozhat, ami pénzügyi rendszerekben komoly probléma lehet.

```
protected $casts = [
    'amount' => 'decimal:2',    // Fee
    'fine'    => 'decimal:2',    // LibraryIssue
];

// MIÉRT NEM FLOAT?
// float esetén:
$ertek = 0.1 + 0.2;    // 0.300000000000000004 ← HIBA!

// decimal:2 esetén:
$fee->amount;    // 15000.50 ← pontos, mindig 2 tizedesjegy
```

4.6 hashed – jelszó titkosítás cast

A hashed cast automatikusan bcrypt algoritmussal titkosítja a jelszót mentéskor. Nem kell manuálisan Hash::make()-et hívni.

```
protected $casts = [
    'password' => 'hashed',    // User
];

// Automatikus titkosítás - mentéskor bcrypt hash lesz:
$user->password = 'titkos123';
$user->save();
// Az adatbázisban: '$2y$12$abc123...' (bcrypt hash)

// FONTOS: a $hidden tömbben is szerepel, hogy API válaszban
// soha ne jelenjen meg a jelszó!
protected $hidden = ['password', 'remember_token'];
```

5. Összefoglaló táblázatok

5.1 ORM kapcsolatok összefoglalója

Kapcsolat	Ökölszabály	Foreign key	Példa a projektből
belongsTo	"Én tartozom valakihez"	Ebben a táblában	Student → SchoolClass (class_id)
hasMany	"Nekem sok van"	A másik táblában	SchoolClass → Students
hasOne	"Nekem pontosan egy van"	A másik táblában	SchoolClass → Timetable

5.2 \$casts típusok összefoglalója

Cast típus	Mire alakítja	Mikor használjuk	Modellek
date	Carbon dátum objektum	Születési dátum, határidők	Student, Teacher, Fee, Exam
datetime	Carbon dátum+idő objektum	Küldési idő, olvasási idő	Notification, NotificationRead
datetime:H:i	Carbon idő H:i formátumban	Órarend, vizsga időpontja	Timetable, Exam, TransportStop
decimal:2	Pontos tizedes szám	Pénzösszegek, bírságok	Fee, LibraryIssue
hashed	Bcrypt hash	Jelszavak	User

6. Hasznos tippek fejlesztés közben

6.1 Kapcsolatok lekérdezése feltétellel

```
// Csak azok a diákok akiknek van legalább 1 vizsgaeredménye:
$students = Student::has('examResults')->get();

// Csak azok akiknek 3-nál több vizsgaeredménye van:
$students = Student::has('examResults', '>=', 3)->get();

// Diákok akiknek van bukott vizsgájuk:
$students = Student::whereHas('examResults', function ($query) {
    $query->where('status', 'fail');
})->get();
```

6.2 Kapcsolaton keresztüli lekérdezés

```
// Egy osztály összes vizsgaeredménye (a diákokon keresztül):
$class = SchoolClass::with('students.examResults')->find(1);

// Egy diák átlaga:
$avg = $student->examResults()->avg('marks_obtained');

// Lejárt díjak lekérdezése:
$overdue = Fee::where('status', 'overdue')
    ->where('due_date', '<', now())
    ->with('student')
    ->get();
```

6.3 Soft Delete hasznos metódusai

```
// Csak a nem törölt rekordok (alapértelmezett):
Student::all();

// Törölt rekordok is:
Student::withTrashed()->get();

// Csak a törölt rekordok:
Student::onlyTrashed()->get();
```



```
// Visszaállítás:  
$student->restore();  
  
// Valódi törlés (véglegesen):  
$student->forceDelete();
```

– School Management App fejlesztési dokumentáció –